

PROJECT REPORT

MACHINE LEARNING



**CARE to Compare: A real-world dataset for
anomaly detection with wind turbine data**

Content Submitted and Authored by

WP1 - Nitesh Morem and Kashif Riyaz

WP2 - Nitesh Morem

Supervised by

Prof. Dr. Patrick Levi

Ostbayerische Technische Hochschule Amberg-Weiden
Department of Electrical Engineering, Media and Computer Science

June 4, 2025

Abstract

This project presents the results of anomaly detection on Wind Farm C using the CARE benchmark. Following the project structure, WP1 covers collaborative data analysis, while WP2 focuses on individual model development. Both supervised and unsupervised learning approaches were explored for predictive maintenance using SCADA data.

1 Introduction

This report evaluates anomaly detection models using SCADA data from Wind Farm C, part of the CARE to Compare benchmark. The dataset includes 58 time-series files from 22 turbines—27 with labeled faults and 31 representing normal operation. Each file contains one year of training data and several days of prediction data, sampled at 10-minute intervals.

The evaluation strictly follows the CARE framework, which scores model performance across four key metrics: Coverage, Accuracy, Reliability, and Earliness.

1.1 Dataset Overview

Each dataset contains up to 957 features, including metadata such as status-type-id, train-test, timestamp, id, and asset-id. The train-test column indicates which samples are for training and prediction. Anomalies are defined based on whether faults appear during the prediction phase of a dataset the that dataset is labeled Anomaly. The asset-id identifies individual turbines, and id denotes sample indices.

1.2 CARE Score Evaluation

To evaluate the model’s anomaly detection performance, the CARE score was used. CARE is a composite metric that captures four key aspects: Coverage, Accuracy, Reliability, and Earliness. The metrics were derived from the original publication; for more details, refer to [1].

- **Coverage (F_β)** measures how many true anomalies were correctly detected, balancing recall and precision using a weighted F-score. It rewards models that detect a high proportion of actual anomalies with minimal false positives[1].

- **Accuracy** evaluates how well the model identifies normal operation by computing the proportion of correctly predicted normal intervals. This helps ensure that the model does not over-flag anomalies in non-faulty periods[1].
- **Reliability (Event F_β)** is calculated at the event level. A true positive event is counted if the model raises any alert during the actual fault window. False positive events are those with no overlap with real faults. This penalizes scattered or spurious anomaly signals[1].
- **Earliness** measures how early an anomaly is detected within its labeled interval. The earlier the signal is raised within the fault window, the better the score. This is computed across all correctly detected events and averaged[1].

These four components are aggregated into a single CARE score, providing a balanced evaluation that considers detection quality, timing, and operational reliability[1].

2 Data Analysis and Feature Engineering (WP1)

Authored jointly by: Nitesh Ramesh Morem and Kashif Riyaz

2.1 Feature Reduction and Selection

There is high dimensional feature space this is because each sensor has average, standard deviation, Minimum, maximum column, below is how we reduced the feature space.

- Anomaly type based feature selection using metadata in (eventinfo.csv).
- Merging of perfectly correlated features using correlation matrix set at 0.9 threshold.
- Selective merging of sensors which measure in same units or same thing while checking the correlation.

The features were reduced with high priority of anomaly based events as described in the metadata (eventinfo.csv) there is clear description of what kind of failure occurred in a dataset. Below in this table are the anomaly types and their related sensors.

Anomaly Types in Datasets	Sensor ids
Hydraulic System	sensor_48–55, sensor_74, sensor_81–82
Pitch/Rotor Brake	sensor_12–14, sensor_54–55, sensor_76, sensor_100–105, sensor_9–11
Gear Oil/Mechanical	sensor_44–45, sensor_87–89, sensor_94, sensor_106, sensor_117–118
Electrical/Converter	sensor_35–38, power_5–6, power_17, sensor_127–131, reactive_power_119–122
Temperature/Environmental	sensor_7, sensor_15, sensor_18–21, sensor_62–65, sensor_74
Communication/Control	sensor_9–15, sensor_100–105
Yaw/Axis	sensor_27–34, sensor_62–64, sensor_100–105

Table 1: Feature groups retained per anomaly type

With this approach, having already known about the anomaly types it makes more sense to take sensors which are related to a certain anomaly and keeping all those sensors. Moving on some of the features were then selectively merged where they were measuring the same thing as there were many sensors which were a backup sensors which were removed using correlation matrix.

2.2 Dropping Min and max of sensors

The sensors having max and min readings were dropped as we realized that we don't know the sampling rate of sensors. As each sample is a 10 minutes interval[1], so only a single max or min value is recorded per interval. This long window means that different samples may share identical mean or standard deviation values but have different min and max values, which will only dilute the model and corrupt the predictive ness of the model.

After filtering we have 240 features after dropping min and max , a reduced set of features was retained with only standard deviation and average for anomaly detection. The features directly related to wind speed and power output were prioritized, as they are critical to identifying deviations from the expected turbine behavior.

2.3 Normalization

All numeric features were standardized using standard scaler normalization to ensure uniform scaling across sensors. This step was essential for models sensitive to feature magnitude. We are basically transforming a column that makes the mean 0 and the standard deviation becomes 1.

3 Methodology – (WP2)

Authored by: Nitesh Ramesh Morem

I used two different approaches to try and evaluate the problem statement which are Supervised and Unsupervised learning.

3.1 Supervised Learning approach

In the supervised learning approach, I selected wind speed and power output as core features, based on visual inspection and their clear importance to turbine behavior. These non-anonymized sensors are consistent across datasets and provide interpretable, generalizable signals for detecting anomalies.

3.1.1 Theoretical Power curve

The theoretical power curve defines the ideal relationship between wind speed and power output under optimal conditions[2]. It is used as a baseline to detect deviations, flagging anomalies when actual turbine output diverges from expected performance.

A typical wind turbine power curve is defined by three characteristic speeds:

- **Cut-in (V_c):** Minimum wind speed to start power generation.
- **Rated (V_r):** Wind speed at which maximum power is produced.
- **Cut-out (V_s):** Maximum wind speed before shutdown to prevent damage.

I used $V_c = 3.0$, $V_r = 12.0$ and $V_s = 25$ for the base dataset 44.csv. Theoretical curves assume ideal conditions, but real-world performance often deviates due to environmental factors, mechanical wear, and control system behavior[2][3].

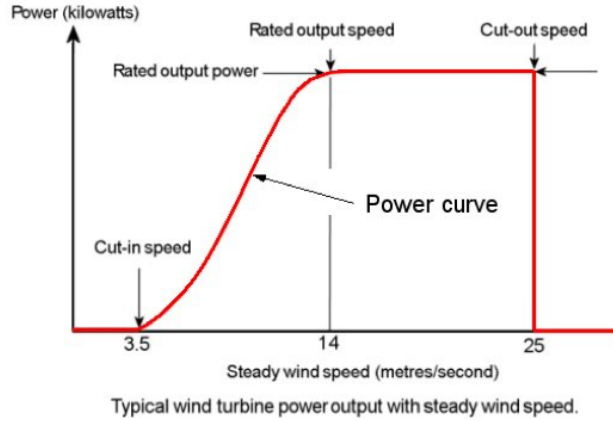


Figure 1: Typical theoretical power curve of a wind turbine

Source: <https://www.kaggle.com/code/winternguyen/wind-power-curve-modeling>

3.1.2 Separating Normal and Anomaly points

I used Theoretical power curve to investigate whether the wind turbines are working in ideal power curve or not. I investigated using status ids provided by the benchmark description . I used only status id's '0' from the train segment of dataset. The reason for selecting only status ID '0' is that anomalies typically do not occur during clearly marked fault periods, but rather during intervals assumed to be normal. and the benchmark datasets clearly states to cross check it with wind and power sensors[1]. By comparing actual performance against the theoretical power curve during these intervals, deviations were identified as potential anomalies even in the absence of fault labels.

DBscan clustering

I used DBscan to just separate the deviations and normal points. My goal was not to get a perfect cluster or good silhouette score but only visually clear separation of deviations.

All the red points are considered abnormal points or the points deviating from the curve as anomaly points and the blue points are Normal points.

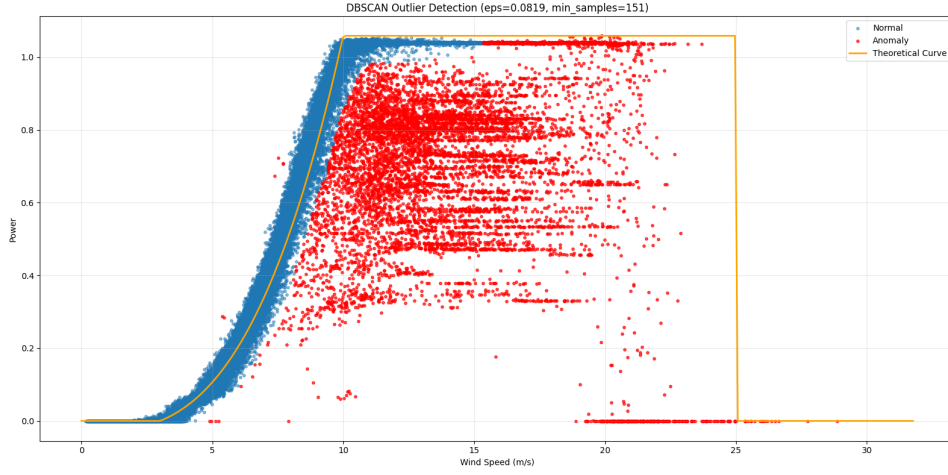


Figure 2: Power curve with status id 0

WHY KDE - Kernel Density Estimation(Gaussian)

I used KDE to investigate the overlapping normal points inside the anomaly points and check the density of the distribution of both normal and anomaly points. KDE was applied to model the distribution of power values during normal turbine operation and also for the anomaly points as shown in the 34.

KDE is particularly useful in this context because it does not assume any fixed distribution shape, making it suitable for real-world sensor data that may not follow ideal statistical patterns. It provides a smooth and interpretable estimate of the data density, allowing for visual and quantitative comparison between normal and anomalous behavior.

Filtering normal points based on Density

I already knew that my clustering is poor so based on density i was able to remove the overlapping points and remove all the points of power greater than 0.99 or 1.0 values from the anomaly points.

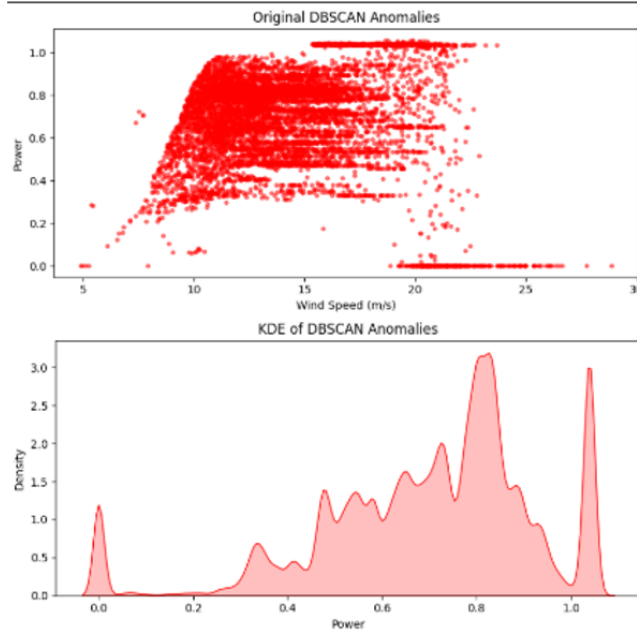


Figure 3: KDE distribution of DBscan anomaly points

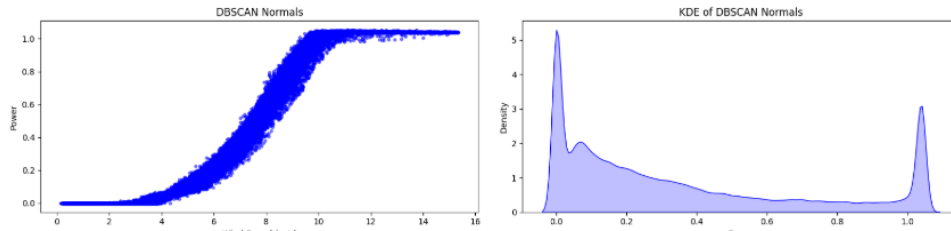


Figure 4: The normal points Distribution

I could have also filtered more anomaly points but the problem is the anomaly points are very less compared to normal points. So i decided to have some less important points, rather than no data points at all. As there is a huge data imbalance when it comes to Normal and Anomaly points.

After looking at the density graphs in Figure 3 the points around 1.0 are overlapping in both Original dbscan based data points and also in normal points in Figure 4. This was crucial as we don't want flag normal points as anomaly.

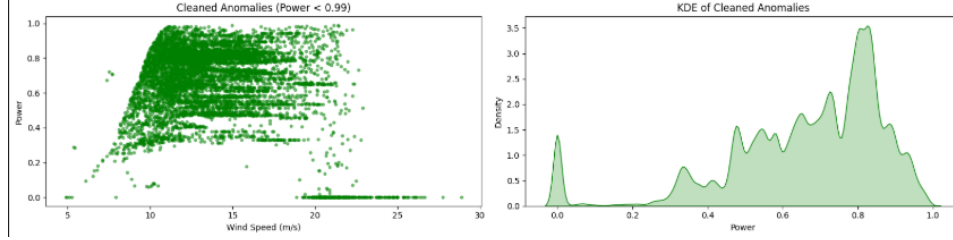


Figure 5: The cleaned points Distribution

In the Figure 5 it can be clearly seen that the overlapping points are removed and only the deviations or anomaly points are retained. when comparing all the three Figures- 345. It can be clearly seen why it was very important to eliminate those points. As the distribution in normal points at 1.0 was removed or cleaned.

3.1.3 Labelling and Train-Test split

The labeling was done based on the refined or cleaned anomaly points from the KDE distribution. There is a column or binary feature in dataset which has value train and prediction. As the benchmark literature clearly stated that if a dataset contains anomaly in the prediction of a dataset, then they labeled it anomaly dataset[1]. so i used the prediction frame as ground Truth. It is very important to note that i was working only with status ids 0 as all the other status ids are irrelevant as the operators already flagged them faults or failure points.

- **Labels :** I took the cleaned anomalies and made a binary column and flagged them 1 and others 0 for normal points.
- **X-train :** After pre-processing train segment of dataset, the feature space as discussed in above sections with only the sensors and removing the metadata, i have 240 features. And also all the values were scaled using standard scaler.
- **X-test:** The prediction frame was used for testing the model and was also preprocessed with same dimensions as X-train without any metadata.
- **y-train :** I used the binary column created from the clean anomalies with 1(Anomaly) and 0 (normal).

- **y-test** : I took the index range given in the event-info(csv) as ground truth and labeled the given range of the anomaly event and flagged all points in this range as 1(Anomaly) and 0 (normal).

3.1.4 Random Forest Model and Hyperparameter

A Random Forest Classifier was used for supervised anomaly detection due to its robustness to noise and high-dimensional inputs. The model was trained with 180 trees and a max depth of 12. Regularization parameters such as `min_samples_leaf = 11`, `min_samples_split = 22`, and `max_features = 'sqrt'` were tuned to prevent overfitting and improve generalization. Class probabilities were obtained using `predict_proba()` to enable custom thresholding.

Thresholding with `predict_proba()`

A custom threshold of 0.6 was applied to the predicted probabilities to reduce false positives and improve early detection . Unlike `predict()`, which returns hard class labels based on a 0.5 cutoff either 0 or 1, `predict_proba()` allows fine control over sensitivity it give the prediction in range of 0 and 1.0 with this i was able to selct high-confidence anomalies[4]. For some of the turbines a lower threshold was used for developing anomalies.

Cross-Validation

To assess model reliability, 5-fold cross-validation was performed using the F1-score. This ensured consistent performance across splits and helped verify that the model was not overfitting. The F1 metric was chosen to balance precision and recall in the context of imbalanced anomaly labels and the results were optimal showing no signs of overfit and in different folds performed consistently .

3.1.5 Supervised CARE Results

The CARE score ranges from 0 (worst) to 1 (best), where higher values indicate better overall anomaly detection performance. It combines four key metrics:

Table 2: Summary of Early Anomaly Predictions (Lead Time with more than 4h)

Dataset	Lead Time	Conf.	Coverage	Accuracy	Reliability	Earliness	CARE Score
44 (benchmark)	0d 4h 30m	0.610	0.222	0.934	0.077	0.045	0.264
49	1d 18h 50m	0.660	0.146	0.938	0.088	0.081	0.266
55	2d 0h 0m	0.960	0.342	0.206	0.056	0.275	0.206
15	0d 19h 0m	0.930	0.483	0.795	0.200	0.140	0.352
67	0d 22h 10m	0.960	0.577	0.689	0.135	0.209	0.364
9	1d 2h 40m	0.450	0.277	0.995	0.385	0.056	0.354
76	1d 21h 50m	0.830	0.044	0.823	0.088	0.048	0.210

The model was trained on Train Frame of dataset 44 with status id 0 on random forest classifier. And then tested on all the prediction frames of the data set.

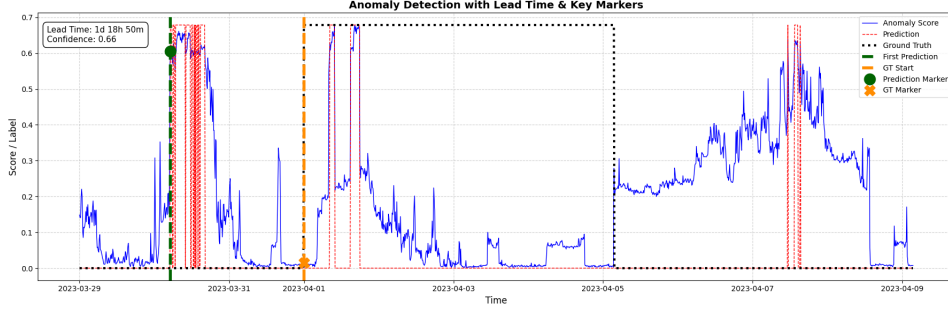


Figure 6: Probability Plot for Dataset 49

The Y- axis shows the probability score where the threshold is 0.6. and the first prediction capture with exact lead time of the first prediction. The threshold 0.6 was decided for most dataset as this threshold the model was able to achieve a predictive maintenance while keeping false positive rate low. The confidence score shows the actual probability score which the model predicted and can be seen as blue lines.

The complete set of results is available in the appendix section (Table 4).

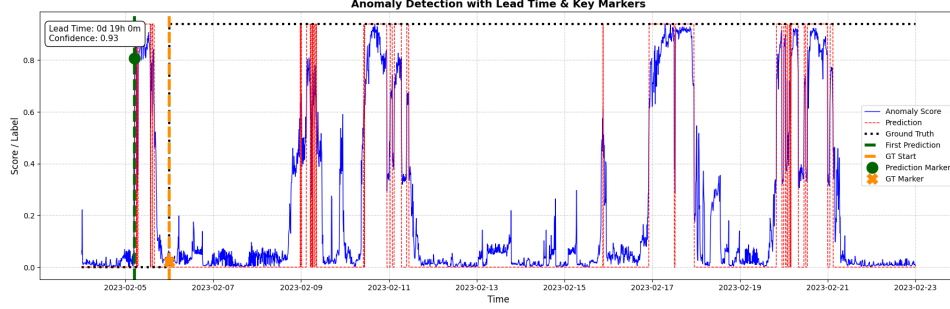


Figure 7: Probability Plot for Dataset 15

3.2 Unsupervised Learning Approach

In the unsupervised approach, clustering was used to detect structural outliers based on the assumption that normal operational data which is status type id = 0 of train data frame has Anomalies in the normal operation of turbine. The process consisted of dimensionality reduction, density-based clustering, and anomaly scoring based on rules :

3.2.1 Downsampling from 10-Minute to Hourly Resolution

The SCADA data was originally recorded at 10-minute intervals, which introduced high-frequency noise and short-term variability. To improve clustering stability and reduce noise, the data was downsampled to hourly resolution by averaging six consecutive samples. Only hours with at least six valid readings were retained.

This step preserved essential trends while reducing fluctuations, improved the reliability of distance metrics in UMAP space, and lowered computational cost without affecting anomaly detection quality.

3.2.2 Hyperparameters Tuning

Hyperparameter tuning for UMAP and HDBSCAN was performed using Optuna, optimizing for silhouette score on non-noise clusters to ensure compact and well-separated groupings.

3.2.3 Dimensionality Reduction with PCA and UMAP

To enhance the robustness of the unsupervised pipeline, two complementary dimensionality reduction techniques were applied sequentially: PCA followed by UMAP.

PCA (Principal Component Analysis) was first used to reduce noise and compress the original high-dimensional feature space into three principal components. PCA efficiently removes redundant variance and stabilizes the structure of the data, which is beneficial before applying more sensitive non-linear methods.

After PCA, **UMAP (Uniform Manifold Approximation and Projection)** was applied for deeper embedding. UMAP preserves the underlying manifold structure of the data, maintaining both local and global relationships. Two embeddings were computed:

- **10D UMAP** for clustering: This higher-dimensional space retains detailed structural information, allowing HDBSCAN to form more coherent and well-separated clusters.
- **3D UMAP** for visualization: This lower-dimensional representation enables intuitive visual inspection of cluster shapes, boundaries, and anomaly separation.

Why PCA Before UMAP and Dimensional Strategy

PCA was first applied to reduce noise and stabilize the high-dimensional feature space by capturing directions of maximum variance. As a linear method, it compresses the data while preserving structure through eigenvalue decomposition[5].

UMAP, a non-linear method, embeds data by preserving neighborhood relationships but relies on Euclidean distances, which become unreliable in very high dimensions. PCA helps mitigate this by making distances more meaningful before UMAP is applied[5].

A 10D UMAP embedding was used for clustering (to retain structural detail), while a 3D embedding was created for visualization. This combination improves clustering quality and interpretability without sacrificing performance.

3.2.4 Distance-Based Anomaly Scoring

For each point assigned to a cluster, its distance to the cluster centroid in 3D UMAP space was computed. A per-cluster distance threshold was defined using the 98th percentile of intra-cluster distances. Points exceeding this threshold, belonging to very small clusters, or located far from any cluster center were flagged as anomalies. This was crucial as feature space has normal points and different normal operating points when the turbine

started and stopped can be varying. Due to different points of normal points and to distinguish between anomalies this scoring is important.

3.2.5 Final Anomaly Detection Rule

A point was marked as anomalous if it satisfied the following:

- It was classified as noise by HDBSCAN.
- Its cluster strength was weak (confidence < 0.3).
- It was far from the centroid of its cluster.
- It belonged to a small cluster (*size* $< 10points$).

An additional **Anomaly Score** was computed as a combination of low cluster confidence and large distance from the cluster center.

3.2.6 Interpretation

This rule-based clustering method is interpretable and well-suited in my case, as it filters out isolated weak signals and focuses on strong deviations in structure. The two clusters are different operating state sand are scatter maybe because of the transitions. As i took probability with respect to the cluster strength and its distance to its centroid and then consider the final predictions as labels. Only the strength of cluster greater than 0.3 was taken and marked as noise. The small and sparse clusters were marked accordingly and all edge cases and mini-clusters were evaluated consistently. This multi-criteria strategy enables robust anomaly detection across varying cluster densities.

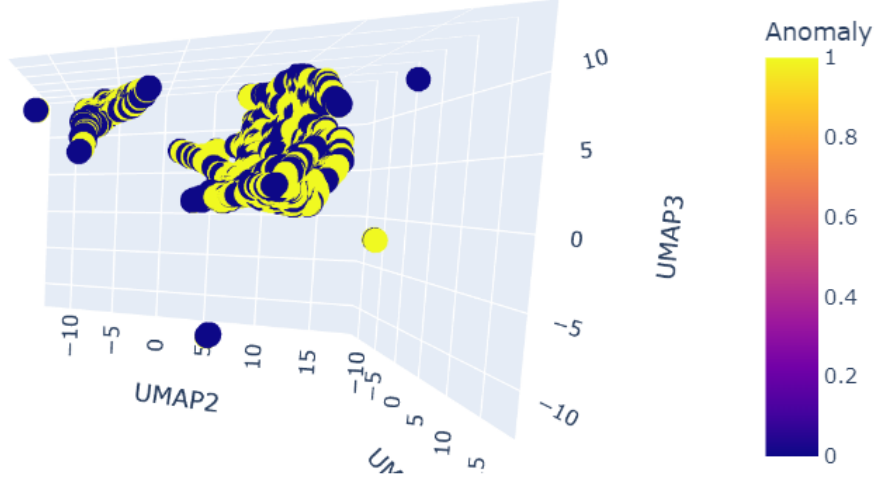


Figure 8: Cluster on train segment + status id 0

3.2.7 Results on Testing Data

For Testing the data , the prediction frame of the dataset was used to test. I used the same hyper-parameters which was used in train frame. In order to get the predictions ,also used anomaly rule base logic discussed in 3.2.5. By applying this logic , I got labels 0 and 1 from clustering and converted these labels into my predictions.

Then i use the index range from the event-info.csv (event-start-id and event-end-id), as we have the ground truth. i then make my true labels according to the index range and mark all the samples with 1 as anomaly and 0 as normal of the prediction frame of all the datasets.

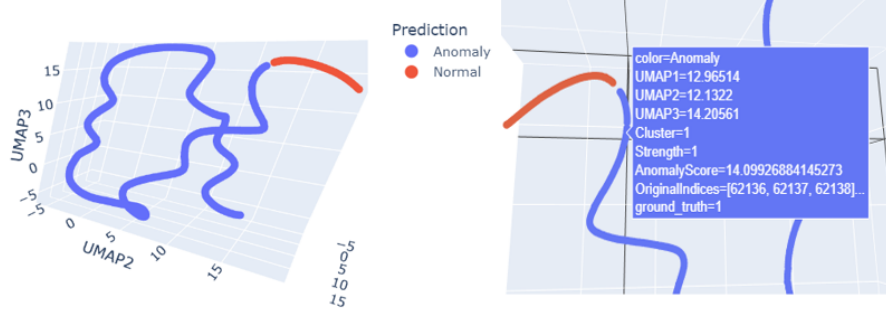


Figure 9: Dataset 44 of Prediction frame(Both images are of same cluster)

Later, i calculated the care score with visual clusters and inspected the generalization. The model performed really good on the benchmark dataset's prediction frame as it was able to correctly separate normal and anomaly points as it can be clearly seen in Figure 9.

Due to the down sampling of the data, the index was transformed , to retain the source index a new column was created to keep track of the respective index and its samples.

The above figure 9 shows the original indices and the original index range given from 52704-62138 index as we can see the normal points are very less and the yellow region is anomaly range,in figure 10 the ground truth range almost cover the whole prediction frame and in the end the normal operating mode which can be correlated in the cluster and cross checked with Clustering with index values . And the clustering algorithm was able to clearly separate two classes. But sometimes it also marks normal points as anomaly. Even after fine tuning the parameters more the result didn't change with any significant, this was the best achievable parameters.

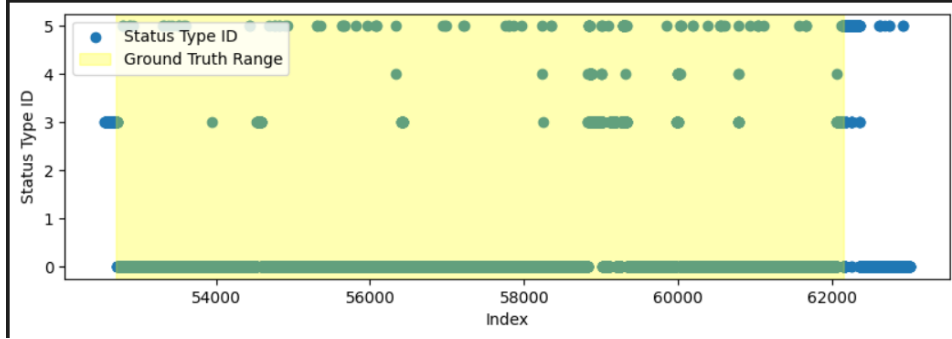


Figure 10: Ground truth range of dataset 44

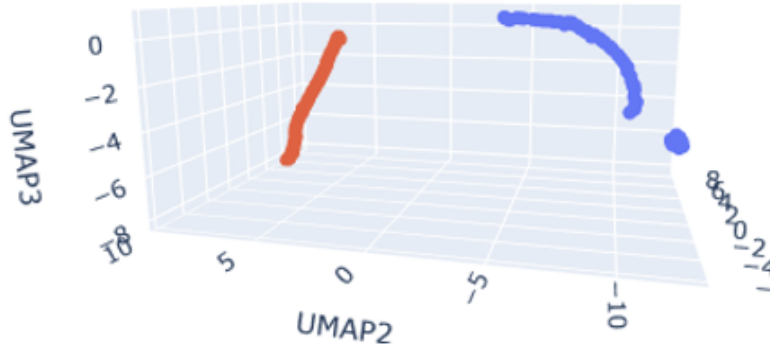


Figure 11: Dataset 76 clustering

Even the clustering in Figure 11 in Dataset 76 , the normal and anomaly class is separated but the problem is the false positive rate in this case is really high and the care score is significantly dropped. It gets very difficult to do any predictive maintenance.

The table below is the evaluation where the clustering method was not able to generalize on all the others but it was able to almost separate normal and anomaly with some errors which lead to poor care scores.

Table 3: CARE Score Summary – Unsupervised Clustering Approach

Dataset	Coverage (F_β)	Accuracy	Reliability (F_β)	Earliness	CARE Score
44 (benchmark)	0.9709	0.6467	1.0000	0.2906	0.6397
81	0.7961	0.0000	1.0000	0.1670	0.0000
47	0.5576	0.1748	1.0000	0.6429	0.1748
79	0.6991	0.6620	0.0000	0.0000	0.2722
76	0.5013	0.5000	0.0000	0.0000	0.2003
5	0.8192	0.0000	1.0000	0.7226	0.0000
31	0.3382	0.2874	1.0000	0.0956	0.2874
90	0.8765	0.5053	1.0000	0.1093	0.5201

4 Conclusion and Future work

Based on experimental results and pipeline stability, the supervised approach is preferred over the unsupervised method.

Supervised learning proved to be more stable and interpretable. It provided consistent performance across datasets and leveraged domain-relevant features (e.g., wind and power sensors), making anomaly predictions more meaningful. The use of label-based evaluation (like the CARE score) also allowed for direct measurement of model effectiveness.

In contrast, the unsupervised approach was highly sensitive to hyperparameters and clustering quality. Methods like HDBSCAN require careful tuning, and their performance varied significantly across datasets. Additionally, unsupervised pipelines often required downsampling to reduce noise, which can lead to loss of temporal resolution and subtle anomalies. Dimensionality reduction (e.g., UMAP) in unsupervised settings was helpful but introduced interpretability challenges.

While unsupervised methods are useful when labels are unavailable, the availability of partial labels and event metadata in this benchmark made the supervised approach both feasible and superior in practice.

References

- [1] C. Gück, C. Roelofs, and S. Faulstich, “Care to compare: A real-world dataset for anomaly detection in wind turbine data,” *arXiv preprint arXiv:2404.10320*, 2024.
- [2] W. Nguyen, *Wind power curve modeling*, <https://www.kaggle.com/code/winternguyen/wind-power-curve-modeling>, Accessed: 2025-06-01, 2022.
- [3] S. Shokrzadeh, M. Jafari Jozani, and E. Bibeau, “Wind turbine power curve modeling using advanced parametric and nonparametric methods,” *IEEE Transactions on Sustainable Energy*, vol. 5, no. 4, pp. 1262–1269, 2014. DOI: 10.1109/TSTE.2014.2345059.
- [4] J. Brownlee. “Threshold moving for imbalanced classification.” Accessed: 2025-06-04. (2020), [Online]. Available: <https://machinelearningmastery.com/threshold-moving-for-imbalanced-classification/>.
- [5] A. T. L. Lun, D. J. McCarthy, and S. C. Hicks, “Dimensionality reduction,” in *Orchestrating Single-Cell Analysis with Bioconductor*, A. T. L. Lun, D. J. McCarthy, and S. C. Hicks, Eds., Accessed: 2025-06-04, Bioconductor, 2022. [Online]. Available: <https://bioconductor.org/books/3.15/OSCA.basic/dimensionality-reduction.html>.

Appendix

Table 4: Complete CARE Evaluation Results Across All Turbines

Dataset	Lead Time	Conf.	Coverage	Accuracy	Reliability	Earliness	CARE Score
81	0d 3h 40m	0.501	0.777	0.916	0.122	0.672	0.632
47	0d 0h 0m	0.000	0.148	0.480	0.046	0.040	0.480
12	0d 0h 0m	0.000	0.106	0.535	0.041	0.025	0.147
4	2d 15h 40m	0.929	0.010	0.816	0.038	0.002	0.173
18	0d 0h 0m	0.000	0.162	0.985	0.122	0.008	0.257
28	0d 8h 40m	0.450	0.292	0.998	0.556	0.091	0.406
39	0d 0h 0m	0.000	0.518	1.000	1.000	0.099	0.543
66	0d 0h 0m	0.000	0.462	1.000	1.000	0.210	0.576
78	0d 0h 0m	0.000	0.033	1.000	1.000	0.010	0.411
79	0d 0h 0m	0.000	0.000	0.965	0.000	0.000	0.000
30	0d 0h 0m	0.000	0.002	1.000	1.000	0.000	0.400
33	0d 0h 0m	0.000	0.370	1.000	1.000	0.114	0.520
11	0d 0h 0m	0.000	0.497	0.990	0.200	0.113	0.383
31	0d 14h 0m	0.489	0.331	0.903	0.200	0.214	0.372
91	0d 8h 30m	0.691	0.290	0.728	0.082	0.064	0.246
5	0d 10h 20m	0.618	0.154	0.965	0.294	0.039	0.298
90	0d 0h 0m	0.000	0.083	0.931	0.082	0.020	0.227
70	0d 0h 0m	0.000	0.601	0.988	0.385	0.246	0.493
35	0d 20h 30m	0.510	0.222	0.993	0.556	0.072	0.383
16	0d 0h 0m	0.000	0.002	1.000	1.000	0.001	0.401

The confidence is the Actual probability score of the first prediction before the ground truth. Also some of the Lead time is 0 this due to be two big reasons which are the prediction frame didn't have much normal points to detect the anomaly and there were also developing anomalies where it was able to detect but the probability i set at high threshold of to avoid false positive rate so the predictions were filtered out. But in this cases it was able it correctly classify anomaly points in the predcitin frame